EDGE INTELLIGENCE-09
NUMBER PLATE DETECTION PROJECT

SUBHASHINI M
25MML0038

1. System Overview

The backend of the Number Plate Detection system is developed using Spring Boot with Java, structured around the MVC (Model-View-Controller) design pattern. Communication between the frontend and backend is handled through REST APIs, which serve as the primary entry points for all incoming client requests.

When a user captures or uploads a vehicle image from the frontend application, the image is transmitted to the backend via an upload API. The controller layer receives the request, validates it, and forwards it to the service layer for further processing.

System Overview Flow

Frontend / Camera System
↓
REST API Endpoint (POST /upload)
↓
Spring Boot Controller Layer
↓
Service Layer (Business Logic)
↓
FTP Server | Detection Model | PostgreSQL Database

2. API Layer — Spring Boot Controller

Spring Boot provides RESTful API endpoints to receive and handle client requests. The Controller layer is responsible for the following:

- Receiving the uploaded image from the frontend
- Validating file format, file size, and request parameters
- Forwarding the validated request to the service layer

API Request Flow

Client Sends: POST /upload (with image file)
↓
Controller Validates: File Format | File Size | Request Parameters
↓
Request Passed to Service Layer

3. Service Layer Processing

The Service class contains the core business logic of the application. After receiving a validated image request, the service layer carries out the following operations in order:

Service Layer Processing Flow

1. Validate the Uploaded Image
↓
2. Generate Metadata (Timestamp, Filename)
↓
3. Verify FTP Connection and Database Availability
↓
4. Transfer Image to FTP Server
↓
5. Trigger Number Plate Detection Model
↓
6. Store Detection Results and Logs in Database

4. Image Storage — FTP Server

Once the image has been validated, it is transferred to a centralized FTP server for storage. This approach provides the following benefits:

- Centralized and secure storage of all uploaded vehicle images
- Reduced load on the main application server
- Efficient file management and easy accessibility for downstream processing

FTP Storage Workflow

Service Layer Initiates FTP Connection
↓
Image Transferred to FTP Server
↓
Stored Data: Image Name | Upload Timestamp | File Path

5. Number Plate Detection — ML Model and OCR

Once the image is stored in the FTP server, the detection pipeline is triggered. The machine learning model processes the image and extracts the vehicle number plate text through the following steps:

Detection Pipeline Flow

Load Image from FTP Server
↓
Run Detection Model on Image
↓
Identify the Number Plate Region
↓
Generate Bounding Box around the Plate Area
↓
Apply OCR to Extract Alphanumeric Characters
↓
Output: Detected Number Plate Text (e.g., KL07AB1234)

6. Database Storage — PostgreSQL

All detection results and system activity logs are stored in a PostgreSQL relational database. The backend uses Hibernate ORM to manage all database interactions, ensuring reliable and efficient SQL operations. The following information is recorded for every request:

Database Fields

Field	Description
filename	Name of the uploaded image file
upload_time	Timestamp of the upload
detected_plate	Extracted number plate text
processing_status	Success or Failure
server_logs	System activity logs

7. Logging and Monitoring

A logging mechanism is implemented to track and monitor all system activities at each stage of processing. Logs are generated for the following events:

- API request received
- Image successfully uploaded to FTP server
- Detection model triggered
- Detection completed or failed
- Error messages for debugging and troubleshooting

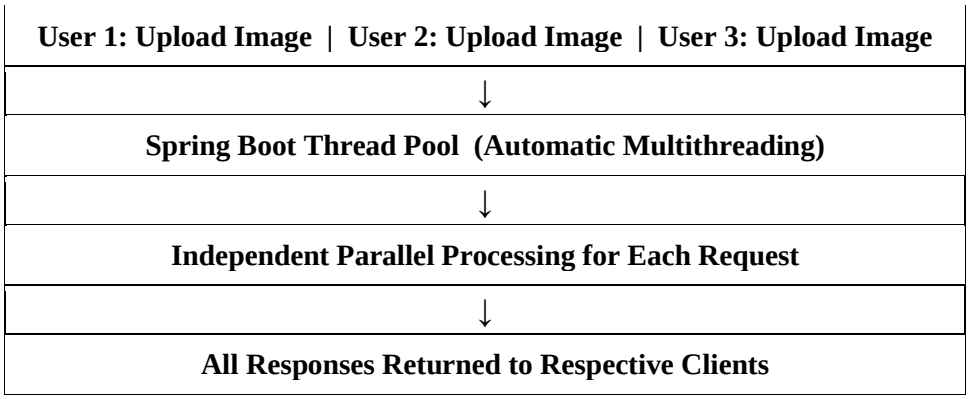
Sample Log Output

```
[INFO]  API request received — POST /upload
[INFO]  Image uploaded successfully — vehicle1.jpg
[INFO]  Detection model triggered
[INFO]  Detection completed — Plate detected: KL07AB1234
```

8. Parallel Processing and Concurrency

Spring Boot supports automatic multithreading, enabling the system to handle multiple simultaneous image upload requests without any performance degradation. Each request is processed independently in its own thread, ensuring no delays even under high load.

Concurrent Request Handling Flow



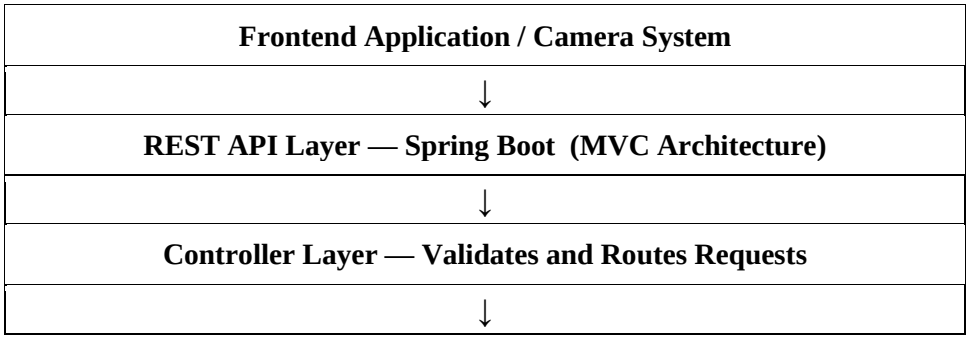
9. API Response to Client

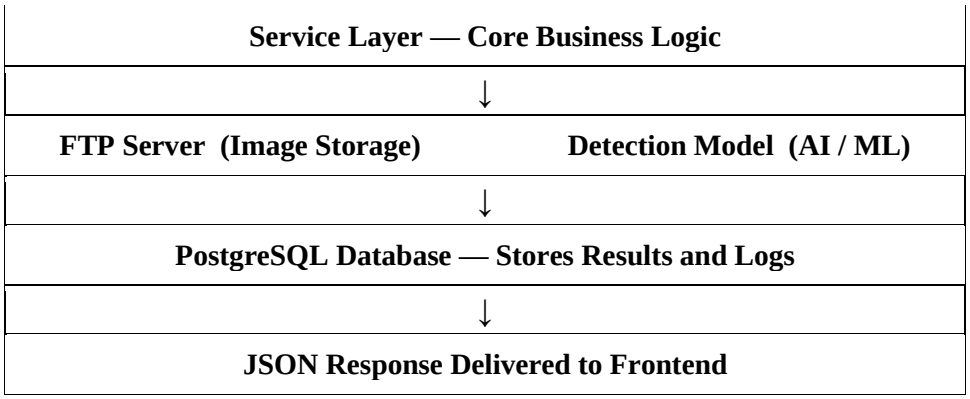
After the detection process is completed, the backend sends a structured JSON response back to the frontend containing the following information:

```
{
    "filename"     : "vehicle1.jpg",
    "plate_number" : "KL07AB1234",
    "status"       : "success",
    "upload_time"  : "2026-03-06 10:21:30"
}
```

10. Complete System Architecture

The following flow represents the complete end-to-end architecture of the Number Plate Detection backend system:





11. Deployment

The complete backend application is packaged as an executable JAR file, making it easy to deploy and run across different environments. The application is started using a single Java command:

```
java -jar number-plate-detection.jar
```

Upon successful startup, the following services are initialized in sequence:

- Spring Boot application server starts and becomes active
- Database connection to PostgreSQL is established
- FTP server connection is configured and verified
- All REST API endpoints become available for incoming requests